

Hamza Qureshi - **Roll No. 04**

Dr. Azam

Advance Analysis of algorithm

09 November 2024

Assignment 02

Use Cases:

- 1. Divide and Conquer in Cluster-Aware Random Stealing (CRS) with WAN**
- 2. Divide and Conquer in Image Compression: Enhancing Data Efficiency**

Merge Sort in Large-Scale Data Processing and Databases

- 3. Binary Search in Search Engines**
- 4. Karatsuba Algorithm: Accelerating Large Multiplications**

Binary Search in the Context of Search Engines (Optimized Querying)

Search engines, like **Google, Bing, or Yahoo**, handle huge amounts of data, processing and indexing millions of web pages to deliver relevant results almost instantly. One critical challenge is to pinpoint the exact location of query results from a vast, organized list of documents or records.

To achieve efficiency, search engines use an *indexing system*. Documents and web pages are sorted by factors such as keyword relevance, frequency, or popularity. When a user searches for a term, the engine uses binary search to quickly locate relevant documents within this sorted index.

Understanding How Search Engines Use Indexes: Search engines build inverted indexes, a data structure where each word maps to a list of documents that contain that word. This structure is similar to a hashmap, where words are keys, and the associated values are document lists.

For example, if you consider the 26 letters of the English alphabet, the URLs or documents that start with each letter are grouped accordingly. However, with large datasets, sorting by document IDs or URLs helps optimize query retrieval.

Types of Inverted Indexes:

1. **Record-Level Inverted Index:** It contains references to the documents where the word appears.
2. **Word-Level Inverted Index:** This includes the specific positions of each word within a document, which requires more space and processing power.

Example of Binary Search with an Inverted Index

Consider an inverted index for the term "machine learning" with a sorted list of document IDs: **[2, 5, 8, 12, 19, 23, 29]**. When a user searches for "machine learning," the search engine:

- Uses binary search to find the term in the sorted document list.
- Start by checking the middle value: for an index range from 0 to 6, the middle index is 3, and the document ID is 12.
- If the query result isn't at 12, the algorithm continues searching in the left or right half of the list until it finds the desired document ID.

This method significantly reduces the number of comparisons, making search operations efficient.

A more simple example is : “hello Hamza, ” “you are writing Advance analysis of algorithm assignment ” “which is rough subject.” now if i index by (text, word within the text) it look like this

→ hello (1, 1)
→ Hamza (1, 2)
→ you (2, 1)
→ are (2, 2)
→ writing (2, 3)
→ Advance (2, 4)
→ analysis (2, 5)

- of (2, 6)
- algorithm (2, 7)
- assignment (2, 8)
- which (3, 1)
- is (3, 2)
- rough (3, 3)
- subject (3, 4)

Inverted Index Construction:

1. **Stop Words Removal:** Common words like "the," "and," and "is" are removed.
2. **Stemming:** Words are reduced to their root forms.
3. **Document ID Assignment:** Each word is mapped to the documents it appears in.

Benefits of Using Binary Search in Search Engines

1. **Efficient Querying:** For a word like "algorithm," the inverted index provides its exact location, allowing quick retrieval.
2. **Handling Large Datasets:** Binary search efficiently handles the massive amount of indexed data.
3. **Optimized Retrieval:** Search engines avoid exhaustive searches, making query results faster.
4. **Support for Complex Queries:** The system can handle multiple and complex search queries efficiently.

Sample Code:

```
function binary_search(arr, start, end, lookUpNum) {  
  if (start > end) {  
    console.info("Number not found.");  
    return false;  
  }  
  
  const MidIndex = Math.floor((start + end) / 2); // 0+8/2=>4 1st time  
  (slice array into half)  
  const MidElem = arr[MidIndex];  
  
  if (lookUpNum === MidElem) {  
    console.info(`Found the number: ${MidElem}`);  
  }  
}
```

```

        return true;
    }

    // Recursively search in the right half
    if (lookUpNum > MidElem) {
        console.info("right side", MidElem);
        return binary_search(arr, MidIndex + 1, end, lookUpNum);
    }

    // Recursively search in the left half
    if (lookUpNum < MidElem) {
        console.info("left side", MidElem);
        return binary_search(arr, start, MidIndex - 1, lookUpNum);
    }
}

const lookingUpThisNumber = -2;
const arr7 = [1, 1, 34, 2, 4, 9, 6, 99, 6.4];
const sorted_arr = bubble_sort(arr7, arr7.length);
binary_search(sorted_arr, 0, sorted_arr.length - 1, lookingUpThisNumber);

// Time Complexity will be:
// fn= O(log n)

```

Merge Sort in Large-Scale Data Processing

In the case of **Databases (indexing)** and **big data frameworks or data warehousing or large log files or large text files** the process of sorting and merging data is crucial. so data typically cannot be processed in a single step due to its size that may exceed the system's memory limits. so the external sorting algorithm (e.g:- merge sort) is the most commonly used one because it divides the data into chunks and processes each chunk separately

Example: Large File Sorting

If a file contains 10 million records but memory can only handle 1 million at a time:

1. **Chunk Division:** Split the file into 10 chunks of 1 million records each.
2. **In-Memory Sorting:** Sort each chunk using algorithms like QuickSort or HeapSort.

3. **Merge Phase:** Recursively merge sorted chunks to form a single sorted dataset.

External Sorting in Distributed Systems

Big Data frameworks like **Apache Hadoop** and **Spark** leverage Merge Sort for efficient data processing. Sorting and merging are fundamental for operations like indexing, duplicate removal, and aggregation.

- **Early Aggregation:** Absorbs duplicate rows in-memory before writing, reducing duplicate data handling.
- **Wide Merging:** Merges multiple runs simultaneously, minimizing the need for repeated merge levels and boosting performance.

Sample Code:

```
function merge_sort(arr) {
  if (arr.length < 2) {
    // console.info("Array less than 2 is already sorted array");
    return arr;
  }

  const MidIndex = Math.floor(arr.length / 2); //slice array into half
  const leftArr = arr.slice(0, MidIndex);
  const rightArr = arr.slice(MidIndex, arr.length);

  return merge(merge_sort(leftArr), merge_sort(rightArr)); // merge is
// O(n) and merge_sort is // O(log n)
}

// compare elements of leftArr with rightArr
function merge(leftArr, rightArr) {
  let resultArr = [];
  let leftIndex = 0;
  let rightIndex = 0;

  while (leftIndex < leftArr.length && rightIndex < rightArr.length) {
    if (leftArr[leftIndex] < rightArr[rightIndex]) {
      resultArr.push(leftArr[leftIndex]);
      leftIndex += 1;
    } else {
```

```

        resultArr.push(rightArr[rightIndex]);
        rightIndex += 1;
    }
}
return resultArr
    .concat(leftArr.slice(leftIndex))
    .concat(rightArr.slice(rightIndex));
}

const arr8 = [1, 1, 34, 2, 4, 9, 6, 99, 6.4];
console.info(merge_sort(arr8));

// Time Complexity will be:
// fn= O(n log n)

```

Karatsuba Algorithm: Fast Multiplication for Cryptography, Scientific Computing, and Big Data

The Karatsuba algorithm is a fast multiplication method that significantly reduces the number of operations needed for multiplying large numbers. It employs a divide-and-conquer strategy, reducing the multiplication complexity to $O(n^{\log_3 2})$ or approximately $O(n^{1.585})$, compared to the traditional $O(n^2)$ method.

Applications of Karatsuba Algorithm

1. **Cryptography:** Used extensively in RSA and other cryptographic algorithms, where fast multiplication of large numbers is crucial.
2. **Scientific Computing:** Optimizes large-scale computations like matrix multiplications in physics simulations or signal processing.
3. **Big Data Processing:** Enhances efficiency in handling operations on large datasets.

How Karatsuba Algorithm Works

The algorithm splits large numbers into smaller parts and applies multiplication in a way that minimizes the number of required operations:

- Consider two numbers a and b , which are split into halves: x_1, x_0 and y_1, y_0 .
- Using the formula:

$$(a+b)(c+d) = ac + bd + (a+b)(c+d) - ac - bd$$

The multiplication reduces to three simpler multiplications instead of four, making the process more efficient.

Benefits in Cryptography

1. **Faster Key Generation:** Critical for RSA, where key generation involves multiplying large primes.
2. **Efficient Encryption/Decryption:** Speeds up operations, vital for real-time applications.

Impact in Scientific Computing

In fields like machine learning and signal processing, Karatsuba's faster multiplication helps with extensive computations, making large-scale simulations more efficient.

Role in Big Data

The algorithm enhances data processing tasks involving large numbers, like hashing and analytics. Faster multiplication reduces computational time, improving overall data processing efficiency.

By reducing the complexity of multiplication-heavy operations, the Karatsuba algorithm remains a powerful tool in optimizing modern computing tasks.

Sample Code:

```
function karatsuba(x, y) {  
  // If either number is a single digit, return the direct multiplication  
  if (x < 10 || y < 10) {  
    return x * y;  
  }  
}
```

```

// Convert numbers to strings for easier manipulation
let xStr = x.toString();
let yStr = y.toString();

// Find the length of the numbers
let n = Math.max(xStr.length, yStr.length);
console.info(n, "max");

// Split the numbers into two halves
let half = Math.floor(n / 2);
console.info(half, "half");

// First half and second half of x and y
let x1 = parseInt(xStr.slice(0, xStr.length - half)); // First half of x
let x0 = parseInt(xStr.slice(xStr.length - half)); // Second half of x
let y1 = parseInt(yStr.slice(0, yStr.length - half)); // First half of y
let y0 = parseInt(yStr.slice(yStr.length - half)); // Second half of y

// Recursively calculate the three products
let z2 = karatsuba(x1, y1); // z2 = x1 * y1
let z0 = karatsuba(x0, y0); // z0 = x0 * y0
let z1 = karatsuba(x1 + x0, y1 + y0) - z2 - z0; // z1 = (x1 + x0) * (y1
+ y0) - z2 - z0
console.info(z1);

// Combine the three products using the Karatsuba formula
return z2 * Math.pow(10, 2 * half) + z1 * Math.pow(10, half) + z0;
}

// Example usage:
let x = 1234;
let y = 5678;

console.log(
  `Karatsuba multiplication result of ${x} and ${y}:`,
  karatsuba(x, y)
);

```


Divide and Conquer in Cluster-Aware Random Stealing (CRS) with WAN

In a parallel computing environment, Divide and Conquer is a method that simplifies a complex problem by breaking it down into smaller, independent subproblems.

This technique involves:

1. Dividing the main problem into smaller subproblems.
2. Assigning each subproblem to a CPU (or a set of CPUs) to solve.
3. Combining the results of the subproblems to obtain the final solution

It minimizes the need for CPUs to interact frequently, enhancing speed and scalability.

CRS (Cluster-Aware Random Stealing) optimizes the task management and load balancing in WAN-connected clusters. The RS (Random Stealing) is inefficient because of,

High Communication Latency: longer time for data to travel

Limited Bandwidth: slower data transfer speed

Inter-Cluster Communication: Transferring tasks across clusters can cause inefficiencies, especially if not handled carefully.

The communication delays can significantly slow down the process.

- ❖ CRS resolves these issues by making clusters "aware" of each other, prioritizing local task-stealing to limit inter-cluster communication.

How Divide and Conquer Powers CRS?

1. **Problem Decomposition:** Divide and Conquer transforms a complex problem into simpler subproblems, which are executed in parallel. This allows CRS to distribute tasks efficiently, leveraging multiple processing units.
2. **Efficient Load Distribution:** It intelligently manages workloads. By prioritizing task distribution within local clusters and minimizing WAN communication, CRS enhances performance and mitigates latency issues.

Task Stealing in CRS

CRS shows a specialized form of task stealing, optimized for WAN constraints:

- Rules and Strategies:
 1. **Selective Stealing:** Only unfinished or uninitiated/uninitialized tasks can be stolen, preventing disruption of ongoing computations.

2. **Targeted Work Stealing:** CPUs strategically target workloads that have accumulated substantial unfinished tasks, avoiding random stealing.
3. **Granularity Consideration:** Small, independent tasks are prioritized for stealing to minimize complexity and dependencies.
4. **Hierarchical Task Management:** Stealing occurs primarily at deeper recursion levels to improve efficiency because tasks are already divided into smaller subproblems.
5. **Queue Management:** Work queues are structured to streamline task distribution, with CPUs stealing tasks from the queue's end to maintain order.

CRS Optimization Techniques

- **Local Priority**
- **Network-Aware Decisions**
- **Intelligent Task Movement**

Benefits of CRS Over Traditional Approaches

- **Reduced Communication Overhead**
- **Enhanced Load Balancing**
- **Resource Optimization**

In case If there are only 2-3 subproblems, but you have many CPUs, then the extra CPUs will just be sitting idle until they can steal tasks. The system will handle this by allowing idle CPUs to grab tasks from others.

Divide and Conquer, when integrated with **CRS**, optimizes parallel task processing in distributed systems. By minimizing cross-cluster communication, CRS manages workloads efficiently, reducing latency and enhancing overall system performance. Its intelligent approach to task stealing makes it superior to traditional load-balancing methods, especially in WAN environments.

Divide and Conquer for Image Compression

There are 2- types of Image compressions:

1. **Lossy Compression:** It reduces the file size by removing data that is less perceivable to the human eye, such as JPEG using **Discrete Cosine Transform (DCT)**. It's suitable when exact data replication is not necessary.
2. **Lossless Compression:** Retains original data, making it reversible. It's preferred for archival purposes, but it achieves less significant size reduction.

The Role of Discrete Cosine Transform (DCT)

Basically DCT is a mathematical technique used to convert spatial domain data into frequency domain data. For image compression, it separates smooth transitions (low frequencies) from sharp edges (high frequencies).

Spatial Domain: In the spatial domain, we deal with the actual pixel values and their locations in the image. For example, RGB pixel [12, 133, 222] at (0, 0) describes a specific color at that location.

Frequency Domain: It Shows how pixel values change over space. **Low frequencies** correspond to smooth areas (e.g., the sky, gradual shading), while **high frequencies** represent abrupt changes (e.g., zebra stripes).

Steps in JPEG Compression Using DCT

1. **Image Division:** Convert the image to grayscale and divide it into 8x8 blocks.
2. **Apply DCT:** Transform each block (8x8) from spatial to frequency domain.
3. **Quantization:** Reduce precision of high-frequency coefficients, which are less noticeable, leading to significant compression.
4. **Zigzag Scan:** Rearrange coefficients to group low frequencies together, aiding in efficient entropy coding.
5. **Entropy Coding:** Compress coefficients using methods like Huffman or Run-Length Encoding.

Divide and Conquer in JPEG Compression

Breaking the image into 8x8 blocks simplifies processing and enables parallelism:

- **Localized Operations:** Each block undergoes independent DCT, quantization, and encoding, reducing computational complexity.
- **Parallel Processing:** Independent blocks allow parallel computation, speeding up the compression process.
- **Efficient Storage:** By handling smaller sections, compression and data management become more efficient.

Consider a large image with dimensions **1024x1024** pixels. Instead of processing the whole image at once, you divide it into smaller **8x8** blocks. This means you have:

- $1024/8=128$ blocks per row
- $1024/8=128$ blocks per column

So you have **$128 \times 128 = 16,384$** . Each of these blocks can be independently compressed and then reassembled, enabling parallel computation. Operations like DCT, quantization, and entropy coding are applied to each block, reducing complexity and computational load.

By focusing on smaller blocks, it allows for targeted precision reduction in high-frequency details (**less important**), leading to better compression,

References:

<https://dl.acm.org/doi/abs/10.1145/379539.379563>

<https://dl.acm.org/doi/pdf/10.1145/3568027>

<https://www.geeksforgeeks.org/inverted-index/>